



پیاده‌سازی، استقرار و تطابق طراحی با کد

سامانه یکپارچه آموزش آنلاین مدارس

فاز ششم: ساخت نمونه اولیه | گزارش تطابق کد با طراحی | تاریخچه کامیت‌های Git

مقدمه فاز ششم

از طراحی مفهومی تا نمونه اولیه قابل اجرا

در این فاز، معماری طراحی شده در مرحله قبل به یک نمونه اولیه قابل اجرا تبدیل شد. هدف اصلی این بود که پیاده‌سازی تا حد امکان با نمودار DCD و معماری انتخاب شده در فاز طراحی منطبق باشد تا علاوه بر اجرای سناریوهای اصلی سیستم، توسعه و نگهداری پروژه نیز حفظ شود.

پیش از شروع کدنویسی، ساختار پوشه‌ها، مسئولیت هر لایه و نحوه ارتباط بین اجزای سیستم مشخص شد و پیاده‌سازی بر همان اساس انجام گرفت.

ساخت نمونه اولیه

تبدیل معماری به کد قابل اجرا مطابق DCD



گزارش تطابق کد با طراحی

بررسی همراستایی UML، الگوها و ساختار کد



تاریخچه کامیت‌های Git

مستندسازی روند توسعه مرحله به مرحله



ساختار پروژه و معماری لایه‌ای

معماری لایه‌ای در کنار اصول Domain Driven Design

`online_education_platform`

`controller`

`domain`

`event`

`repository`

`service`

`strategies`

`main.py`

`README.md`



Domain

موجودیت‌ها و قوانین اصلی کسب‌وکار — بدون وابستگی به پایگاه‌داده یا رابط کاربری



Service

پیاده‌سازی منطق اجرایی سیستم و فرایندهای کسب و کار



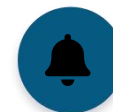
Repository

مسئول مدیریت و نگهداری داده‌ها



Controller

مدیریت درخواست‌های ورودی سیستم



Event

مدیریت رویدادها و ارسال اعلان‌ها

محیط توسعه و فرآیند Build

زبان، معماری و مراحل ساخت نمونه اولیه

محیط توسعه

زبان برنامه‌نویسی Python 3



معماری شیء‌گرا



اجرای پروژه از طریق Python Interpreter



عدم نیاز به نصب پایگاه داده



استفاده از In-Memory Repository برای نگهداری داده در حافظه



مراحل فرآیند Build

دریافت سورس پروژه

1

قرارگرفتن فایل‌ها در ساختار تعریف‌شده

2

ایجاد نمونه Repositoryها

3

ایجاد سرویس‌های دامنه

4

تزریق وابستگی‌ها

5

مقداردهی اولیه سیستم

6

اجرای سناریوی آزمایشی

7

مدیریت وابستگی‌ها و لایه دامنه

Domain و موجودیت‌های Dependency Injection

تزریق وابستگی (Dependency Injection)



کلاس‌هایی مانند ExamService به‌طور مستقیم Repository ها را ایجاد نمی‌کنند؛ بلکه Repository های موردنیاز هنگام ایجاد شیء به آن‌ها تزریق می‌شوند. همین رویکرد در CourseService و NotificationService نیز رعایت شده است.

کاهش وابستگی مستقیم میان کلاس‌ها



ساده‌تر شدن تست سیستم



امکان جایگزینی Repository یا پایگاه‌داده بدون تغییر منطق



موجودیت‌های لایه Domain

تمامی موجودیت‌ها از کلاس BaseEntity ارث‌بری می‌کنند و شناسه یکتای هر شیء با UUID تولید می‌شود.

User

Student

Teacher

Admin

Course

ClassSession

Enrollment

Content

Exam

Question

Submission

Grade

Notification

پیاده‌سازی Repository و منطق کسب‌وکار

الگوی Repository و لایه Service

لایه Repository



برای هر موجودیت اصلی یک Interface مستقل تعریف و سپس نسخه In-Memory آن پیاده‌سازی شد. داده‌ها در Dictionary نگهداری می‌شوند و در آینده می‌توان بدون تغییر سایر بخش‌ها، به پایگاه داده‌هایی مانند PostgreSQL یا MySQL متصل شد.

- UserRepository
- EnrollmentRepository
- SubmissionRepository
- ClassSessionRepository
- CourseRepository
- ExamRepository
- NotificationRepository

لایه Service



منطق اصلی سیستم در این لایه قرار دارد و مسئول اجرای فرآیندهایی مانند ثبت نام دانشجو، ایجاد آزمون، ارسال پاسخ، محاسبه نمره، مدیریت اعلان‌ها و ارائه محتوای آموزشی است.



CourseService



ExamService



GradeService



NotificationService

الگوهای طراحی پیاده‌سازی شده

پیاده‌سازی کامل الگوهای GoF انتخاب‌شده در فاز معماری



Repository

جداسازی منطق ذخیره‌سازی داده‌ها از منطق کسب‌وکار



Strategy

روش‌های مختلف تصحیح آزمون — انتخاب تصحیح خودکار یا دستی



Decorator

اعمال جریمه تأخیر بدون تغییر منطق اصلی تصحیح، افزودن درصد جریمه به نمره



Observer

مدیریت اعلان‌ها و رویدادها؛ انتشار مستقل رویداد و واکنش سرویس‌های مختلف

استقرار (Deployment) و سناریوی آزمایشی

اجرای Local و سناریوی تست سرتاسری سیستم

مراحل استقرار

1 دریافت سورس پروژه

2 اجرای فایل main.py

3 ایجاد تمامی Repositoryها

4 ایجاد سرویسها

5 ایجاد داده‌های اولیه

6 اجرای سناریوی تست

سناریوی آزمایشی سرتاسری

ایجاد یک استاد و یک دانشجو

ثبت یک درس جدید

ثبتنام دانشجو در درس

ایجاد و انتشار آزمون

ارسال پاسخهای دانشجو

نمایش نمره، وضعیت قبولی و اطلاعات ورود کاربر

این سناریو نشان می‌دهد اجزای مختلف سامانه مطابق طراحی اولیه با یکدیگر تعامل دارند.

ساختار نهایی پروژه در یک نگاه

تطابق کامل با ساختار طراحی شده در فاز معماری

```
online_education_platform
├── controller
│   ├── __init__.py
│   ├── app_controller.py
│   └── exam_controller.py
├── domain
│   ├── base_entity.py · class_session.py · course.py
│   ├── enrollment.py · exam.py · notification.py · user.py
├── event
│   ├── event_listeners.py · event_publisher.py
├── repository
│   ├── in_memory_repo.py · interfaces.py
├── service
│   ├── strategies: auto.py · base.py · manual.py · late_penalty.py
│   ├── course_service.py
│   ├── exam_service.py · grade_service.py · notification_service.py
├── main.py
└── README.md
```

گزارش تطابق کد با طراحی

بررسی همراستایی طراحی UML و پیاده‌سازی نهایی سیستم

تطابق ساختار کلی پروژه

سیستم در طراحی به‌صورت ماژولار در نظر گرفته شده بود و پیاده‌سازی نیز مطابق همین رویکرد توسعه یافته است.

`create initial project structure`

ایجاد ساختار اولیه پروژه

`Create .gitkeep`

ایجاد ساختار استاندارد پوشه‌ها

`promote Enrollment to dedicated feature module`

انتقال ماژول ثبت‌نام به ساختار اختصاصی

هدف این گزارش، بررسی میزان هماهنگی میان طراحی سامانه آموزش آنلاین مدارس (UML)، مدل‌سازی موجودیت‌ها و ساختار ماژول‌ها (و پیاده‌سازی نهایی سیستم در بخش کد است. بررسی تاریخچه تغییرات در Git نشان می‌دهد توسعه پروژه مرحله‌ای بوده و هر بخش از طراحی در مراحل مختلف به کد منتقل شده است.

تطابق مدل دامنه و الگوهای طراحی با کد

از موجودیت‌های مفهومی و الگوهای GoF تا کلاس‌های واقعی

بخش طراحی	پیاپی سازی در کد	کامیت مرتبط
موجودیت پایه سیستم	BaseEntity	<code>implement BaseEntity class</code>
موجودیت آزمون	Exam Entity	<code>implement exam domain entities</code>
جلسه کلاس	ClassSession Entity	<code>reformat ClassSession entity</code>
ثبت نام دانش آموز	Enrollment Module	<code>promote Enrollment to dedicated feature module</code>
الگوی Strategy برای تصحیح آزمون	Strategy Pattern (auto/manual)	<code>implement base strategies</code>
الگوی Observer برای اعلان ها	Observer Pattern (Event)	<code>Observer ساختار و بازسازی مرتبط با</code>

تطابق کنترلر، Repository و مستندات با طراحی

کنترلر



کنترلرها مسئول دریافت درخواست‌ها و ارتباط میان رابط کاربری و منطق سیستم بودند؛ این جداسازی مسئولیت‌ها در کد نیز رعایت شده است.

`created app controller`

Repository



الگوی Repository برای جدا کردن منطق کسب‌وکار از نحوه ذخیره‌سازی داده‌ها پیاده‌سازی شد و اینترفیس‌های آن بهبود یافتند.

`improve interfaces code`

مستندات UML



نمودارهای UML همراه با کد تکمیل شدند تا طراحی رفتاری سیستم با فرآیند توسعه هماهنگ بماند.

`Create Activity Diagrams`

`Create Sequence diagram`

نتیجه‌گیری تطابق طراحی و کد

بررسی روند توسعه نشان می‌دهد پیاده‌سازی انجام‌شده تطابق مناسبی با طراحی اولیه سیستم دارد. ساختار ماژولار، موجودیت‌های دامنه، کنترلرها، اینترفیس‌ها، Repository ها و الگوهای طراحی تعریف‌شده در مرحله طراحی، در کد نهایی نیز پیاده‌سازی شده‌اند.

معماری طراحی‌شده



در ساختار کد پیاده‌سازی شده است

مدل دامنه



به کلاس‌های نرم‌افزاری تبدیل شده است

الگوهای طراحی



در بخش‌های مختلف سیستم استفاده شده‌اند

مستندات طراحی



همراه با توسعه کد تکمیل شده‌اند

در مجموع، میزان تطابق میان طراحی و کد پروژه بالا ارزیابی می‌شود.

تاریخچه کامیت‌های Git

مدیریت نسخه‌ها و مستندسازی روند توسعه پروژه



github.com/jafaripourmotahareh/Shad-Education-Platform

Shad-Education-Platform

تعداد **Commit** ها قبل از فاز ۱۲۸۶ :

برای مدیریت تغییرات، نگهداری نسخه‌های مختلف و پیگیری روند توسعه، از Git و مخزن GitHub استفاده شد. ثبت منظم تغییرات نقش مهمی در مستندسازی روند توسعه و هماهنگی میان اعضای تیم ایفا کرده است.

اهداف استفاده از Git

مدیریت نسخه‌های مختلف پروژه



ثبت تغییرات انجام‌شده در هر مرحله



امکان بازگشت به نسخه‌های قبلی



مدیریت همکاری میان اعضای گروه



نگهداری تاریخچه توسعه پروژه



افزایش قابلیت ردیابی تغییرات



جلوگیری از از بین رفتن اطلاعات



کامیت‌های کلیدی و مهم پروژه

گزیده‌ای از کامیت‌های فنی و معنادر روند توسعه

#	پیام Commit	شرح فعالیت
1	<code>create initial project structure</code>	ایجاد ساختار اولیه پروژه
2	<code>implement BaseEntity class</code>	پیاده‌سازی کلاس پایه موجودیت‌ها
3	<code>implement exam domain entities</code>	پیاده‌سازی موجودیت‌های دامنه آزمون
4	<code>implement base strategies</code>	پیاده‌سازی استراتژی‌های پایه
5	<code>Create Activity Diagrams</code>	افزودن نمودارهای Activity
6	<code>created app controller</code>	ایجاد کنترلر اصلی سیستم
7	<code>improve interfaces code</code>	بهبود کیفیت کد اینترفیس‌ها
8	<code>promote Enrollment to dedicated feature module</code>	انتقال Enrollment به ماژول اختصاصی
9	<code>Create Sequence diagram</code>	افزودن نمودار توالی
10	<code>reformat ClassSession entity</code>	بازآرایی موجودیت جلسه کلاس

تحلیل و جمع‌بندی کامیت‌ها

روند تکاملی توسعه از ساختار اولیه تا معماری کامل

مستندسازی تغییرات

تمامی تغییرات به‌صورت منظم در Git ثبت شده‌اند. تاریخچه ثبت‌شده نشان می‌دهد توسعه پروژه مرحله‌ای انجام شده و تمامی اصلاحات و به‌روزرسانی‌ها در قالب Commit های متوالی ثبت شده‌اند.

افزایش نظم در مدیریت پروژه

امکان رهگیری تغییرات

بازیابی نسخه‌های قبلی

هماهنگی مؤثر میان اعضای گروه

روند تکاملی توسعه

ایجاد ساختار اولیه و مدل‌سازی موجودیت‌های اصلی

تکمیل زیرساخت‌های نرم‌افزاری (Repository)، (Controller)

پیاده‌سازی الگوهای طراحی (Observer)، (Strategy)

بهبود ساختار کد، بازآرایی ماژول‌ها و کلاس‌ها

جمع‌بندی نهایی فاز ششم

نمونه اولیه قابل اجرا



پیاده‌سازی کامل مطابق DCD با معماری لایه‌ای و اصول DDD

تطابق بالا با طراحی



مدل دامنه، الگوهای GoF و مستندات UML همگی در کد پیاده شده‌اند

الگوهای طراحی فعال



Observer، Decorator، Strategy، Repository به صورت کامل اعمال شدند

مستندسازی کامل توسعه



128 کامیت پیش از فاز ۶ با روند مرحله‌ای و قابل ردیابی در Git